

Over-the-Air Edge Inference for Low-Altitude Airspace: Generative AI-Aided Multi-Task Batching and Beamforming Design

Yang Fu¹, Peng Qin¹, *Member, IEEE*, Yifei Wang, Liming Chen, Mengyao Li²,
and Xiongwen Zhao¹, *Senior Member, IEEE*

Abstract—The exploitation of low-altitude (LA) airspace is advancing globally, boosting the sensing demands for heterogeneous flying aircraft. To fulfill an accurate and intelligent sensing, the future 6G base station (BS) requires to aggregate features of multiple sensors' views, then perform edge inference via loading the artificial intelligence (AI) model. However, this process confronts communication and computation bottlenecks owing to high-dimensional feature uploading as well as frequent memory access. To overcome these bottlenecks, we propose a multi-task over-the-air edge inference system for LA airspace, where feature aggregation is efficiently achieved employing over-the-air computation, and multiple inference tasks arriving at the BS are processed in batches to reduce memory access. Under this arrangement, we formulate a joint batching and beamforming design problem to maximize the number of completed tasks, constrained by completion latency and inference accuracy requirements. To address this intractable problem, we first examine the case with synchronous task arrivals and single batch. A spatial correlation-aware beamforming design approach is proposed to effectively suppress feature aggregation error and ensure inference accuracy. Next, we delve into the general asynchronous task arrival case. An AI-generated online policy is developed, which innovatively utilizes diffusion model to output batching decisions, thereby adapting to the dynamic and uncertain nature of task arrivals. Simulation results obtained on real-world dataset corroborate the importance of capturing the spatial correlation among sensors. In addition, the proposed approach realizes outstanding performance compared to benchmark batching, beamforming, and learning methods, and the completed task amount is close to offline optimization with prior task information.

Index Terms—Low-altitude airspace, edge inference, batching, over-the-air computation, multi-antenna beamforming, diffusion model.

Received 17 December 2024; revised 16 March 2025; accepted 18 April 2025. Date of publication 23 April 2025; date of current version 20 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62201212, 62271201, in part by the Natural Science Foundation of Hebei Province under Grant F2022502017, in part by the Fundamental Research Funds for the Central Universities under Grant 2023JC003, and in part by the Science and Technology Projects of China Southern Power Grid under Grant SEPRI-K24B018. The associate editor coordinating the review of this article and approving it for publication was M. Chen. (*Corresponding author: Peng Qin.*)

Yang Fu, Peng Qin, Yifei Wang, Mengyao Li, and Xiongwen Zhao are with the State Key Laboratory of Alternate Electrical Power System with Renewable Energy Sources, School of Electrical and Electronic Engineering, North China Electric Power University, Beijing 102206, China (e-mail: qinpeng@ncepu.edu.cn).

Liming Chen is with the Electric Power Research Institute, China Southern Power Grid, Guangzhou 510663, China.

Digital Object Identifier 10.1109/TCOMM.2025.3563657

I. INTRODUCTION

A. Background

LOW-altitude (LA) airspace refers to the physical space that accommodates the flight operations of various aircraft, including electric vertical take-off and landing, autonomous aerial vehicles (AAVs) and so on [1]. In recent years, the exploitation of LA airspace is advancing globally, with the aim of promoting substantial economic growth and unlocking new commercial applications, e.g., air transport, AAV delivery, and city monitoring [2]. As predicted by the Federal Aviation Administration of USA, advanced air mobility will be officially commercialized between 2025 and 2027, and the number of flights will increase to 3.9 million per year by 2030 [3]. However, the ever-increasing of flying aircraft calls for not only full coverage, but also high-precision and intelligent sensing, fulfilling the safety supervision of LA airspace [4]. This necessitates the paradigm shift from conventional communication systems to multi-functional information networks.

As a milestone in the development of communication systems, the Recommendation for IMT-2030 defines two new functions of 6G, i.e., sensing and artificial intelligence (AI) [5]. The former leverages edge devices as distributed sensors to achieve collaborative sensing, offering multi-view observations of surrounding environment. For the latter, advanced AI models are deployed at network edge in order to empower ubiquitous inference on sensing data. The natural integration of these two new functions, which combines the advantages of collaborative sensing and the powerful inference abilities of AI models, gives birth to an intelligent platform for performing diversified inference tasks of LA airspace, incorporating aircraft recognition, auto-pilot and autonomous control [6], [7].

Such an intelligent platform builds on the popular split inference architecture, in which a pre-trained AI model is partitioned into two sub-models [8]. Each LA sensor utilizes the lightweight sub-model to extract local features from its raw observation, then wirelessly uploads the features to a base station (BS) for aggregation and inference via loading the edge sub-model. Evidently, this inference architecture allows the resource-limited sensors to access the high-performance edge AI model, while improving inference accuracy through aggregating features extracted from various sensors' views. Despite these strengths, realizing efficient edge inference in practical LA airspace still confronts two major bottlenecks. Specifically, communication bottleneck stems from the transmission of

high-dimensional features from numerous LA sensors to the BS. Meanwhile, computation bottleneck is caused by the time-consuming memory access for AI model loading, enlarging the edge inference latency [9]. To tackle these bottlenecks, related studies start to explore the power of emerging techniques, e.g., over-the-air computation (AirComp) and batching.

B. Related Works and Motivation

1) *Work in Over-the-Air Edge Inference:* AirComp allows multiple sensors to simultaneously upload analog signals over the same wireless medium, then the BS can directly receive aggregated signals thanks to the wave superposition property [10]. As such, rapid and scalable feature aggregation can be achieved, thereby overcoming the communication bottleneck. Considering an edge inference system that aggregates multi-view observations via AirComp, paper [11] constructed a theoretical framework for characterizing the inference uncertainty. The analysis results unveiled an exponential decrease of inference uncertainty as the number of observations ascends. The authors of [12] proposed a generalized configuration scheme to calculate various aggregation functions, e.g., max and average, based on AirComp. Subsequently, configuration parameter and sensor power were jointly designed to minimize the aggregation error. In [13], a task-oriented optimization approach for over-the-air edge inference was developed, in which an approximate and tractable inference accuracy metric, namely discriminant gain, was derived. Afterwards, the authors systematically designed data sensing, feature extraction and aggregation to maximize the discriminant gain. Literature [14] investigated an edge inference architecture building on cloud radio access network, where the local features were first aggregated at multiple remote radio heads (RRHs) using AirComp, then further transmitted to a central processor, enhancing the network coverage and inference accuracy. However, the aforementioned studies all concentrate on the case of single inference task, which implies that only one common target can be sensed. This encounters difficulties in fulfilling the connected intelligence for LA airspace, since a large number of aircrafts operate simultaneously, resulting in the co-existence of multiple inference tasks. The consideration of multi-task edge inference triggers the first open question motivating our research *Q1: How to carry out reasonable transceiver design for AirComp-based feature aggregation, suppressing the inevitable inter-task interference?*

2) *Work in Multi-Task Batching:* Batching serves as an effective technique for breaking the computation bottleneck, designed to process multiple inference tasks in parallel by assembling them into one batch, thereby amortizing the memory access time and boosting the completed task amount [15]. Under the context of cloud computing, the authors of [16] designed an elastic batch scheduling policy, which dynamically configured the batch size and waiting time to enhance task responsiveness. Work in [17] elaborated an edge inference system with batching capable BS, for which an energy minimization problem was formulated subject to task latency constraints. To solve this problem, the authors first optimized task offloading and computational resource allocation, then batched tasks with similar latency constraints. Literature [18] designed multi-task batching together with

early exiting, which allowed each batched task to traverse an adaptive depth of AI model according to its accuracy demand. Under this arrangement, a tree-search algorithm with online pruning was developed to jointly determine batching and bandwidth assignment. Considering the heterogeneous arrivals of various inference tasks, [19] proposed to form multiple batches and appropriately design the task-batch association as well as batch starting instants, thereby maximizing the inference throughput while respecting the completion deadline and radio resource restraints. In spite of these advancements, a common limitation in these works is that they rely on the precise prediction of task information (containing arrival time and deadline). In practice, such information may not be obtained prior to the batching design, necessitating an online policy that maintains a good balance between immediate task processing and resource reserved for uncertain future arrivals. This gives rise to the second research question *Q2: How to coordinate multi-task batching under information uncertainty, meeting the distinct inference accuracy and completion latency requirements?*

3) *Work in Generative AI-Aided Resource Allocation:* The emergence of generative AI has underscored its creative and versatile content generation capabilities, enabling highly intelligent human-computer interactions [20]. As a representative technology in the realm of generative AI, diffusion model notably excels in modeling complex distributions and generating high-quality data [21]. These unique characteristics make the applications of diffusion model go beyond the original image generation field and extend to network resource allocation. Study [22] employed diffusion model in closed-box optimization scenarios, in which the denoising process of diffusion model was exploited to iteratively generate solutions. In [23], diffusion model was integrated with deep reinforcement learning (DRL), forming an effective decision-making approach to solve the dynamic task assignment problem. Literature [24] investigated the transmitter resource allocation issue in semantic communication scenarios, and applied diffusion model to characterize the importance of user semantic information whilst generating power allocation decisions, improving the reliability of semantic information transmissions. The authors of [25] formulated a joint AI model splitting, task offloading, and resource allocation problem in intelligent vehicular network, for which a multi-agent DRL algorithm was enhanced by diffusion model, achieving precise action value estimation. Nonetheless, the study on diffusion model-aided resource allocation is still in its nascent stage. Different from previous works, we explore a new application of diffusion model in multi-task batching for edge inference system, entailing judicious Markov decision process (MDP) formulation and diffusion-based actor network design. Additionally, we examine the performance gap between diffusion model-based online policy and mathematical optimization with prior task information, offering insights into the implementation of diffusion model in dynamic and uncertain environment.

C. Contributions

In this paper, we put forward an edge inference system for LA airspace, where sensors monitor the behaviors of heterogeneous aircraft and generate multiple inference tasks. An edge

BS aggregates the features extracted from various sensors' views via AirComp, then reconstructs global features for different tasks. Employing the batching technique, the global features are strategically assembled and fed into the edge AI model for parallel inference. We formulate a joint batching (incorporating number of batches, task-batch association, and batch starting time) and beamforming (for AirComp-based feature aggregation) design problem to maximize the number of completed tasks, constrained by completion latency and inference accuracy requirements. Considering the stochastic arrivals and distinct requirements of inference tasks, we tackle this intractable problem by sequentially treating two cases with incremental complexity, namely one presumes synchronous task arrivals and single batch, and the other considers general asynchronous task arrivals. This allows the former's method for reducing the feature aggregation error to be a cornerstone for the batching design in the latter. Our contributions are summarized below.

- To answer *Q1*, we propose to exploit the multiple-input multiple-output (MIMO) technology for inter-task interference suppression in over-the-air edge inference, where the LA sensors and BS are equipped with multiple antennas. After relating the inference accuracy and feature aggregation error, we derive a closed-form expression for the latter to facilitate MIMO beamforming design. Particularly, our beamforming design captures the feature correlation among sensors, bringing performance gain in reducing the aggregation error. Moreover, batching for synchronous arrivals is optimized by finding the completable task subset with maximum cardinality.
- To answer *Q2*, we develop an AI-generated online batching policy to adapt to the dynamic and uncertain nature of task arrivals. First, the coordination of multi-task batching is reformulated as an MDP, in which the action is designed to balance the completed task amount, waiting latency, and BS's busy time. Second, we construct an actor network to generate action based on diffusion model, enabling progressive action refinement through the denoising process. Finally, the diffusion-based actor network is trained to yield batching decisions, in response to asynchronous task arrivals and distinct completion deadlines.
- Comprehensive simulations are conducted to evaluate the proposed algorithms. Leveraging real-world dataset and AI model architecture, we verify the existence of feature correlation among sensors, demonstrating the effectiveness of the proposed correlation-aware beamforming design. In addition, we compare the training convergence of our approach with standard DRL methods, showcasing the superiority of diffusion-based actor network in handling the stochastic and uncertain task arrivals. Besides, numerical results unveil that our online batching policy achieves comparable task completion performance with much lower implementation time, compared to the baseline optimization relying on prior task information.

Notations: The conjugate transpose, transpose, and conjugate operation are denoted by superscripts H , T , and $\dagger$, respectively. For a vector $\mathbf{f}$, $\|\mathbf{f}\|$ represents its norm, $f[m]$ is the m -th element, and $\mathbf{f}[1:m]$ indicates a sub-vector with

TABLE I
SUMMARY OF MAIN NOTATIONS

Notation	Description
N	Number of inference tasks
K_n	Number of LA sensors associated with task n
T	Number of communication rounds
$\mathbf{f}_{n,k}, \tilde{\mathbf{f}}_{n,k}$	Original and normalized feature vector of sensor $\langle n, k \rangle$
$2Q$	Number of feature elements
$M^{\text{tx}}, M^{\text{rx}}$	Number of antennas at LA sensor and BS
$\mathbf{w}_{n,k}$	Transmission beamforming vector of sensor $\langle n, k \rangle$
$\mathbf{H}_{n,k}$	MIMO channel between sensor $\langle n, k \rangle$ and the BS
$\mathbf{v}_n$	Receiving beamforming vector for task n
η_n	Normalizing factor for task n
$\xi_{n,i}$	Signal overlapping indicator for task n and task i
$\hat{\mathbf{f}}_n, \mathbf{f}_n$	Reconstructed and ideal global feature for task n
$\mathbf{e}_n$	Feature aggregation error for task n
T_n, D_n	Arrival instant and completion deadline of task n
B	Number of batches
$y_{n,b}$	Batching decision that determines whether to assemble task n in batch b
t_b, l_b	Starting time and processing latency of batch b
A_n	Inference accuracy requirement of task n
$\mathbf{C}_n$	Feature correlation matrix of task n
$\zeta, \zeta^{\max}$	Index and total number of time slots
U	Number of denoising steps
θ, φ	Parameters of diffusion-based actor and critic networks

elements indexed from 1 to m . For a matrix $\mathbf{F}$, $f[m, n]$ and $\mathbf{f}[m]$ are the (m, n) -th element and m -th column, respectively. $\mathbb{R}^{M \times N}$ and $\mathbb{C}^{M \times N}$ respectively specify the space of $M \times N$ real and complex matrices. $\mathbf{0}_{M \times N}$ and $\mathbf{1}_{M \times N}$ denote the $M \times N$ all-zero and all-one matrices, and $\mathbf{I}_M$ is the $M \times M$ identity matrix.

II. SYSTEM MODEL

As depicted in Fig. 1, we consider a multi-task over-the-air edge inference model for LA airspace. There are K LA sensors responsible for monitoring the behaviors of heterogeneous aircraft, whilst generating data samples for AI inference. A typical sensing task supported by the considered system is LA object recognition,¹ where the BS analyzes the image or video clip to identify the type of object, e.g., suspicious AAV. Let $\mathcal{K}_n = \{1, \dots, K_n\}$ represent the set of K_n LA sensors, which collaboratively perform inference task $n, n \in \mathcal{N} = \{1, \dots, N\}$ by perceiving a common aircraft from multiple views, and $K = \sum_{n \in \mathcal{N}} K_n$. To complete the inference tasks, each sensor first utilizes its local AI model to extract a feature vector from raw data, thereby reducing communication overhead and protecting privacy. Afterwards, leveraging the AirComp technique, all features are aggregated at a BS to reconstruct global features for different tasks. Finally, N global features are input into the edge model equipped by BS to derive the inference results, during which batching is exploited to enhance the inference efficiency. Table I summarizes the main notations used throughout the paper. Detailed models and metrics are illustrated in the sequel.

A. Over-the-Air Feature Aggregation Model

For LA sensor k associated with task n (which is indexed by $\langle n, k \rangle$ in the remainder paper for brevity), the feature

¹A straightforward way to extend the considered system to other types of inference tasks is deploying the corresponding AI models. This does not affect our primary focus, i.e., feature transmission and batched inference.

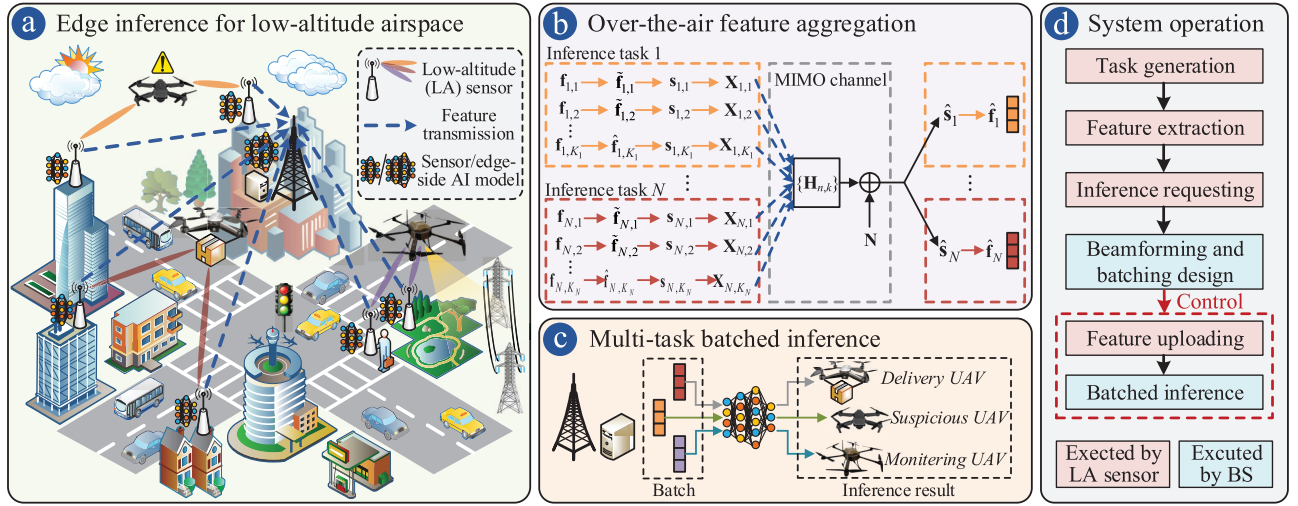


Fig. 1. Multi-task over-the-air edge inference model for LA airspace.

vector $\mathbf{f}_{n,k} \in \mathbb{R}^{2Q \times 1}$ is pre-processed to generate transmission signals, where $2Q$ denotes the number of feature elements. Specifically, $\mathbf{f}_{n,k}$ is normalized as

$$\tilde{\mathbf{f}}_{n,k} = (\mathbf{f}_{n,k} - f_{n,k}^{\text{ave}} \mathbf{1}_{2Q \times 1}) / \sqrt{v_{n,k}}, \quad (1)$$

where $f_{n,k}^{\text{ave}} = \frac{1}{2Q} \sum_{q=1}^{2Q} f_{n,k}[q]$ and $v_{n,k} = \frac{1}{2Q} \sum_{q=1}^{2Q} (f_{n,k}[q] - f_{n,k}^{\text{ave}})^2$ are the mean and variance of $2Q$ elements in $\mathbf{f}_{n,k}$, respectively, and these small-sized scalars can be transmitted to the BS in advance for feature receiving [26]. Then, $\tilde{\mathbf{f}}_{n,k}$ is modulated into analog symbols, i.e.,

$$\mathbf{s}_{n,k} = \tilde{\mathbf{f}}_{n,k} [1 : Q] + j\tilde{\mathbf{f}}_{n,k} [Q + 1 : 2Q] \in \mathbb{C}^{Q \times 1}. \quad (2)$$

By utilizing transmission beamforming vector $\mathbf{w}_{n,k}$, the transmitted signal of sensor $\langle n, k \rangle$ is given by

$$\mathbf{X}_{n,k} = \mathbf{w}_{n,k} \mathbf{s}_{n,k}^T \in \mathbb{C}^{M^{\text{rx}} \times Q}, \quad (3)$$

where M^{rx} is the number of antennas at LA sensors. Moreover, the BS is outfitted with M^{rx} antennas.

Thereafter, $\{\mathbf{X}_{n,k} : \forall n, k\}$ is transmitted over a MIMO channel and received by the BS. Particularly, the receiving beamforming vector $\mathbf{v}_n$ is invoked to aggregate all symbols $\{\mathbf{s}_{n,k} : \forall k \in \mathcal{K}_n\}$ for task n , yielding

$$\hat{\mathbf{s}}_n = \eta_n \mathbf{v}_n^H \left(\sum_{k \in \mathcal{K}_n} \mathbf{H}_{n,k} \mathbf{w}_{n,k} \mathbf{s}_{n,k}^T + \sum_{i \in \mathcal{N} \setminus \{n\}} \xi_{n,i} \sum_{j \in \mathcal{K}_i} \mathbf{H}_{i,j} \mathbf{w}_{i,j} \mathbf{s}_{i,j}^T + \mathbf{N} \right) \in \mathbb{C}^{1 \times Q}, \quad (4)$$

where η_n denotes the normalizing factor, which is applied to scale the received signal for computing the average of sensors' signals. $\mathbf{H}_{n,k} = \sqrt{v d_{n,k}^{-\alpha}} \tilde{\mathbf{H}}_{n,k} \in \mathbb{C}^{M^{\text{rx}} \times M^{\text{tx}}}$ denotes the MIMO channel between sensor $\langle n, k \rangle$ and the BS, where v is the channel gain at reference distance of 1 m, $d_{n,k}$ represents the transmission distance, α indicates the pathloss exponent. $\tilde{\mathbf{H}}_{n,k}$ is the Rayleigh fading component, where each element obeys the circularly symmetric complex Gaussian (CSCG)

distribution with unit variance.² $\xi_{n,i} \in \{0, 1\}$ is the signal overlapping indicator, where $\xi_{n,i} = 1$ implies that the sensors associated with task i , i.e., $\mathcal{K}_i$, simultaneously transmit features with $\mathcal{K}_n$, leading to inter-task interference at the BS, otherwise $\xi_{n,i} = 0$. $\mathbf{N} = [\mathbf{n}[1], \dots, \mathbf{n}[Q]] \in \mathbb{C}^{M^{\text{rx}} \times Q}$ is the channel noise with power σ^2 .

Based on $\hat{\mathbf{s}}_n$, the BS reconstructs the global feature vector for task n as follows

$$\hat{\mathbf{f}}_n = \frac{1}{K_n} [\text{Re}\{\hat{\mathbf{s}}_n\}, \text{Im}\{\hat{\mathbf{s}}_n\}]^T + \frac{\sum_{k \in \mathcal{K}_n} f_{n,k}^{\text{ave}}}{K_n} \mathbf{1}_{2Q \times 1}. \quad (5)$$

To promote further analysis, it is necessary to define the ideal error-free version of $\hat{\mathbf{f}}_n$ as $\mathbf{f}_n = \frac{1}{K_n} \sum_{k \in \mathcal{K}_n} \mathbf{f}_{n,k}$. Consequently, the error caused by over-the-air feature aggregation for task n is denoted as $\mathbf{e}_n = \hat{\mathbf{f}}_n - \mathbf{f}_n$.

B. Multi-Task Batched Inference Model

Due to the high mobility and dynamics of aircraft, inference tasks arrive at LA sensors at random time instants with distinct latency requirements. Let T'_n and D_n denote the arrival instant and completion deadline of task n , respectively. The total latency of each task comprises of 1) feature extraction latency at the sensor, 2) feature transmission latency, and 3) edge inference latency, where the first two parts are constants determined by sensor computing hardware as well as analog symbol transmission.³ Therefore, we can focus on the time instant that each task n arrives at the BS, denoted by T_n .

In the edge inference phase, the BS divides N arriving tasks into B batches (whose set is $\mathcal{B} = \{1, \dots, b, \dots, B\}$) and sequentially processes them. The batching decision is indicated by binary variable $y_{n,b} \in \{0, 1\}$, where $y_{n,b} = 1$ means task

²We utilize the Rayleigh channel model to characterize the non-line-of-sight (NLoS) signal propagation. Please note that our following analysis and proposed approach can be directly extended to other channel models, which will be illustrated in the experiments.

³The transmission latency is $\frac{Q}{W}$ regardless of the number of sensors, where W denotes the bandwidth. Besides, time synchronization in feature transmission can be realized by existing mechanism, e.g., timing advance in 4G Long Term Evolution [26].

n is included in batch b , otherwise $y_{n,b} = 0$. Since each task can only be executed once, we have constraint

$$\sum_{b \in \mathcal{B}} y_{n,b} \leq 1, \quad \forall n, b, \quad (6)$$

and $\sum_{b \in \mathcal{B}} y_{n,b} = 0$ implies that task n is not responded. Meanwhile, the BS determines the starting time of each batch b , denoted by t_b .

To process batch b , the BS assembles the global feature vectors $\{\hat{\mathbf{f}}_n\}$ with $y_{n,b} = 1$ as a high-dimensional tensor [15], which is then fed into the edge AI model for parallel inference. According to [17] and [19], the edge inference latency can be modeled as a linear increasing function with respect to the batch size, i.e.,

$$l_b = \rho_1 + \rho_2 \sum_{n \in \mathcal{N}} y_{n,b}, \quad (7)$$

where ρ_1 and ρ_2 are positive constants, then the completion instant of batch b is given by $t_b + l_b$. Since a batch can start after the arrivals of all contained tasks, as well as the completion of its previous batch, we have

$$t_b \geq \max \left\{ \max_{n \in \mathcal{N}} \{y_{n,b} T_n\}, t_{b-1} + l_{b-1} \right\}, \quad \forall b. \quad (8)$$

C. Performance Metrics

We consider two main performance metrics of AI inference, i.e., completion latency and inference accuracy.

- 1) *Completion Latency*: Each inference task n is deemed successful only if the completion time does not exceed its deadline D_n . As a consequence, we impose the following completion latency constraint:

$$y_{n,b} (t_b + l_b) \leq D_n, \quad \forall n, b. \quad (9)$$

- 2) *Inference Accuracy*: The well-trained AI model possesses inference accuracy of A_0 , which may be degraded due to the over-the-air feature aggregation. To theoretically characterize this, we introduce the classification margin γ that describes model robustness [27]. Concretely, if the feature aggregation error satisfies $\|\mathbf{e}_n\|^2 < \gamma^2$, then the correct inference result can also be derived. Thus, the inference accuracy of task n is lower bounded as

$$A_0 \Pr \left\{ \|\mathbf{e}_n\|^2 < \gamma^2 \right\} \stackrel{(a)}{\geq} A_0 \left(1 - \mathbb{E} \left[\|\mathbf{e}_n\|^2 \right] / \gamma^2 \right), \quad (10)$$

where (a) follows the Markov's inequality. Suppose that the accuracy requirement of task n is A_n , then we can obtain the following constraint on feature aggregation error:

$$\sum_{b \in \mathcal{B}} y_{n,b} \mathbb{E} \left[\|\mathbf{e}_n\|^2 \right] \leq \gamma^2 \left(1 - \frac{A_n}{A_0} \right), \quad \forall n. \quad (11)$$

To glean design insights for the considered edge inference system, we first investigate the case with synchronous task arrivals and single batch in Section III, i.e., $T_1 = T_2 = \dots = T_N = T$ and $B = 1$, which simplifies the batching design and facilitates inference accuracy analysis in terms of over-the-air feature aggregation. Building on the preceding analysis, we further extend to the general asynchronous task arrival case in

Section IV, where a total of N tasks consecutively arrives at the BS at different time instances.

Remark 1 (Relationship Between Synchronous and Asynchronous Task Arrivals as well as Their Practical Applications): Mathematically, the solution for synchronous task arrivals can be regarded as a special case of the asynchronous scenario, with a simplified assumption on T_n . Nevertheless, both of them can be mapped to practical applications in LA systems. Synchronous task arrivals correspond to time-triggered sensing, where all LA sensors detect objects and generate inference tasks at uniform time intervals. Asynchronous arrivals align with event-triggered sensing, where edge inference is triggered by specific events, such as a sensing scheduling, resulting in dynamic and stochastic task generation [28]. Therefore, we develop efficient solutions for both cases.

Remark 2 (Operation of the Considered Edge Inference System): As illustrated in Fig. 1 (d), when a task is generated, the LA sensor first sends an inference request to the BS, including the task's arrival time, deadline, and accuracy requirements. Based on the received task information, the BS implements the beamforming and batching design algorithm in Section III-IV. The resulting decisions are then applied to control the subsequent feature uploading and batched inference processes.

III. EDGE INFERENCE WITH SYNCHRONOUS TASK ARRIVALS AND SINGLE BATCH

In this section, we concentrate on the synchronous task arrivals, where all LA sensors simultaneously transmit features to the BS, and the BS assembles as many tasks as possible into a single batch for edge inference. For this case, an inference throughput maximization problem is formulated, followed by a designed algorithm that achieves efficient batching design and spatial-correlation aware beamforming.

A. Problem Formulation

Our objective is to maximize the inference throughput, i.e., number of completed tasks, under the latency and accuracy requirements. The designed variables are batching strategy⁴ $\mathbf{y} = \{y_n : \forall n\}$ as well as beamforming for transmission $\mathbf{w} = \{\mathbf{w}_{n,k} : \forall n, k\}$ and receiving vector $\mathbf{v} = \{\eta_n, \mathbf{v}_n : \forall n\}$. Consequently, the optimization problem is formulated as

$$\mathbf{P1} : \max_{\mathbf{y}, \mathbf{w}, \mathbf{v}} \sum_{n \in \mathcal{N}} y_n, \quad (12a)$$

$$\text{s.t. } y_n \left(\rho_1 + \rho_2 \sum_{n \in \mathcal{N}} y_n \right) \leq D_n - T, \quad \forall n, \quad (12b)$$

$$y_n \mathbb{E} \left[\|\mathbf{e}_n\|^2 \right] \leq \gamma^2 \left(1 - \frac{A_n}{A_0} \right), \quad \forall n. \quad (12c)$$

$$y_n \in \{0, 1\}, \quad \forall n, \quad (12d)$$

$$\|\mathbf{w}_{n,k}\|^2 \leq P, \|\mathbf{v}_n\| = 1, \quad \forall n, k, \quad (12e)$$

where constraints (12b) and (12c) follow (9) and (11), respectively. (12d) indicates that the batching variables are binary. In (12e), the transmission power of each sensor is restrained by P , while the receiving beamforming vectors are normalized.

⁴Since there is one batch, index b is omitted and starting time $t_b = T$ in this section.

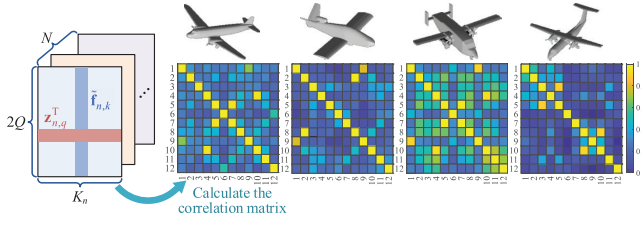


Fig. 2. Statistical results of $\mathbf{C}_n$ using ModelNet dataset, where $c_n[k, j]$ is calculated by $\frac{1}{2Q} \sum_{q=1}^{2Q} \tilde{\mathbf{f}}_{n,k}[q] \tilde{\mathbf{f}}_{n,j}^H[q]$.

Proposition 1: Problem **P1** is NP-hard.

Proof: Please refer to Appendix A⁵. ■

Therefore, **P1** is intractable due to the coupling between binary batching variables and high-dimensional beamforming vectors, and the expectation contained in $\mathbb{E}[\|\mathbf{e}_n\|^2]$ further complicates the problem. To this end, we first convert **P1** into a series of feasibility issues, then determine batching and beamforming by addressing these issues sequentially.

B. Problem Transformation and Batching Design

To tackle **P1**, we find that the $\sum_{n \in \mathcal{N}} y_n$ in (12a) and (12b) only depends on the number of the batched tasks. This motivates us to define $\mathcal{S} = \{n | n \in \mathcal{N}, y_n = 1\}$, $|\mathcal{S}| = \sum_{n \in \mathcal{N}} y_n$, and transform **P1** into finding subset $\mathcal{S} \subseteq \mathcal{N}$ that meets the latency and accuracy constraints with given $|\mathcal{S}|$. Mathematically,

$$\mathbf{P2} : \text{find } \mathcal{S} \subseteq \mathcal{N}, \quad (13a)$$

$$\text{s.t. } y_n (\rho_1 + \rho_2 |\mathcal{S}|) \leq D_n - T, \quad \forall n, \quad (12c) \sim (12e). \quad (13b)$$

As such, we sequentially address **P2** for $|\mathcal{S}| = 1, 2, \dots$ until the largest $|\mathcal{S}|^*$ is found, which makes $|\mathcal{S}|^* + 1$ an infeasible solution of **P2**.

In order to determine the batching result with given $|\mathcal{S}|$, we can easily derive the set $\mathcal{T} = \{n | n \in \mathcal{N}, (\rho_1 + \rho_2 |\mathcal{S}|) \leq D_n - T\}$ that satisfies (13b). If $|\mathcal{T}| < |\mathcal{S}|$, then **P2** is infeasible. For $|\mathcal{T}| \geq |\mathcal{S}|$, we sort the tasks in $\mathcal{T}$ in ascending order based on the accuracy requirement $\{A_n\}$ since the tasks with lower A_n are more likely to be completed, and batch the first $|\mathcal{S}|$ tasks in $\mathcal{T}$ to obtain $\mathcal{S}$. Accordingly, the remaining difficulty lies in optimizing the beamforming vectors, as elaborated in the next subsection.

C. Spatial-Correlation Aware Beamforming Design

Note that in previous works [11], [13], the features of various sensors associated to a common task are assumed to be independent, thereby simplifying the beamforming design. Nevertheless, since the features are extracted from different sensor views of the same target, their spatial correlation may not be ignored. To corroborate this, Fig. 2 shows the statistical results of feature correlation matrix using ModelNet dataset, where $K_n = 12$ LA sensors adopt the local sub-model to extract features from 12 views of a common aircraft,

⁵The appendix is available in our online supplemental material at <https://pan.baidu.com/s/1yvVKBfPehdownYdBhYnRJQ?pwd=ii7t>

and more detailed experiment setup will be presented in Section V. In this figure, features from two sensors have strong spatial correlation when the corresponding pixel is light. We examine most of the aircraft in the dataset (only four examples are shown), and spatial correlation exists in general.

To theoretically capture this, define the q -th column of matrix $[\tilde{\mathbf{f}}_{n,1}, \dots, \tilde{\mathbf{f}}_{n,K_n}]^T$ as $\mathbf{z}_{n,q}$, as illustrated in Fig. 2, and we make the following common assumptions [12]: for each task n , the sensors associated to the task have identical feature variance $v_{n,1} = \dots = v_{n,K_n} = v_n$, and different feature dimensions $\{\mathbf{z}_{n,q} : \forall q\}$ are independent and identically distributed. Then, the feature correlation matrix of task n is given by

$$\mathbf{C}_n = \mathbb{E}[\mathbf{z}_{n,q} \mathbf{z}_{n,q}^H] \in R^{K_n \times K_n}, \quad \forall q, \quad (14)$$

where the (k, j) -th entry of $\mathbf{C}_n$ is represented by $c_n[k, j]$, measuring the spatial correlation between sensors $\langle n, k \rangle$ and $\langle n, j \rangle$. Subsequently, we have the following theorem to characterize $\mathbb{E}[\|\mathbf{e}_n\|^2]$.

Theorem 1: The mean square feature aggregation error of task n is jointly caused the misalignment of intra-task feature aggregation, inter-task interference, and channel noise, i.e.

$$\begin{aligned} \mathbb{E}[\|\mathbf{e}_n\|^2] &= \frac{1}{K_n^2} \sum_{q=1}^Q \left\{ \underbrace{\mathbb{E} \left[\left| \sum_{k \in \mathcal{K}_n} (\eta_n \mathbf{v}_n^H \mathbf{H}_{n,k} \mathbf{w}_{n,k} - \sqrt{v_n}) s_{n,k}[q] \right|^2 \right]}_{\text{Misalignment error}} \right. \\ &\quad + \underbrace{\mathbb{E} \left[\left| \eta_n \mathbf{v}_n^H \sum_{i \in \mathcal{N} \setminus \{n\}} \xi_{n,i} \sum_{j \in \mathcal{K}_i} \mathbf{H}_{i,j} \mathbf{w}_{i,j} s_{i,j}[q] \right|^2 \right]}_{\text{Inter-task interference}} \\ &\quad + \underbrace{\mathbb{E} \left[\left| \eta_n \mathbf{v}_n^H \mathbf{n}[q] \right|^2 \right]}_{\text{Noise}} \left. \right\} \stackrel{(a)}{\geq} \frac{Q v_n}{K_n^2} \left\{ 2 \sum_{k,j \in \mathcal{K}_n} c_n[k, j] \right. \\ &\quad \left. - \frac{\left[\sum_{k,j \in \mathcal{K}_n} c_n[k, j] \left(\mathbf{v}_n^H \mathbf{H}_{n,k} \mathbf{w}_{n,k} + (\mathbf{v}_n^H \mathbf{H}_{n,j} \mathbf{w}_{n,j})^H \right) \right]^2}{2 \sum_{i \in \mathcal{N}} \xi_{n,i} \sum_{k,j \in \mathcal{K}_i} c_i[k, j] \mathbf{v}_n^H \mathbf{H}_{i,k} \mathbf{w}_{i,k} (\mathbf{v}_n^H \mathbf{H}_{i,j} \mathbf{w}_{i,j})^H + \sigma^2} \right\}, \end{aligned} \quad (15)$$

where (a) is deduced by substituting the following optimal η_n^* into $\mathbb{E}[\|\mathbf{e}_n\|^2]$:

$$\eta_n^* = \frac{\sqrt{v_n} \sum_{k,j \in \mathcal{K}_n} c_n[k, j] \left(\mathbf{v}_n^H \mathbf{H}_{n,k} \mathbf{w}_{n,k} + (\mathbf{v}_n^H \mathbf{H}_{n,j} \mathbf{w}_{n,j})^H \right)}{2 \sum_{i \in \mathcal{N}} \xi_{n,i} \sum_{k,j \in \mathcal{K}_i} c_i[k, j] \mathbf{v}_n^H \mathbf{H}_{i,k} \mathbf{w}_{i,k} (\mathbf{v}_n^H \mathbf{H}_{i,j} \mathbf{w}_{i,j})^H + \sigma^2}. \quad (16)$$

Proof: Please refer to Appendix B. ■

Theorem 1 facilitates the beamforming design in two key aspects. First, it establishes a direct link between the expected feature aggregation error and the beamforming vectors $\mathbf{w}, \mathbf{v}$ by providing a closed-form expression. Second, it derives the optimal normalizing factor η_n^* for receiving resultant information uncertainty makes beamforming.

With given $\mathcal{S}$ and let $\xi_{n,i} = y_n y_i$, we recast **P2** as the following beamforming problem with an explicit objective:

$$\mathbf{P3} : \max_{\mathbf{w}, \mathbf{v}, \{\chi_n \geq 0: \forall n \in \mathcal{S}\}} \sum_{n \in \mathcal{S}} \chi_n, \quad (17a)$$

$$\text{s.t. } f_n - \delta_n g_n \geq \chi_n, \quad \forall n \in \mathcal{S}, \quad (17b)$$

By substituting (15) into (12c), (17b) is deduced as a more tractable form of the accuracy constraint, where $f_n^2 = \left[\sum_{k,j \in \mathcal{K}_n} c_n[k,j] \left(\mathbf{v}_n^H \mathbf{H}_{n,k} \mathbf{w}_{n,k} + (\mathbf{v}_n^H \mathbf{H}_{n,j} \mathbf{w}_{n,j})^H \right) \right]^2$, $g_n = 2 \sum_{i \in \mathcal{N}} \xi_{n,i} \sum_{k,j \in \mathcal{K}_i} c_i[k,j] \mathbf{v}_n^H \mathbf{H}_{i,k} \mathbf{w}_{i,k} (\mathbf{v}_n^H \mathbf{H}_{i,j} \mathbf{w}_{i,j})^H + \sigma^2$, $\delta_n = 2 \sum_{k,j \in \mathcal{K}_n} c_n[k,j] - \frac{K_n^2}{Q v_n} \gamma^2 \left(1 - \frac{A_n}{A_0} \right)$. $\{\chi_n : \forall n \in \mathcal{S}\}$ are slack variables that facilitate the inference accuracy of the batched tasks to be higher than their requirements. However, constraint (17b) is still non-convex due to the coupling between $\mathbf{w}$ and $\mathbf{v}$, we optimize them in an alternative manner to derive the sub-optimal solution as follows.

1) *Transmission Beamforming Optimization*: For sensors associated to unbatched task $n \notin \mathcal{S}$, we directly set $\mathbf{w}_{n,k} = \mathbf{0}_{M^u \times 1}$, $\forall k \in \mathcal{K}_n$. Then **P3** reduces to

$$\mathbf{P4} : \max_{\mathbf{w}_{n,k}, \{\chi_n \geq 0: \forall n \in \mathcal{S}\}} \sum_{n \in \mathcal{S}} \chi_n, \quad (18a)$$

$$\text{s.t. } f_n - \delta_n g_n \geq \chi_n, \quad \forall n \in \mathcal{S}, \quad (18b)$$

$$\|\mathbf{w}_{n,k}\|^2 \leq P, \quad (18c)$$

where (18b) can be recast as

$$\begin{aligned} & -\mathbf{w}_{n,k}^H \mathbf{A}_{n,n,k} \mathbf{w}_{n,k} + \underbrace{\left[2\text{Re} \{ \mathbf{b}_{n,k}^H \mathbf{w}_{n,k} \} + \omega_{n,k}^{(1)} \right]^2}_{\triangleq \psi_{n,k}(\mathbf{w}_{n,k})} \\ & - 2\text{Re} \{ \mathbf{c}_{n,n,k}^H \mathbf{w}_{n,k} \} - \omega_{n,k}^{(2)} \geq \chi_n, \quad (19a) \\ & - \xi_{l,n} \mathbf{w}_{n,k}^H \mathbf{A}_{l,n,k} \mathbf{w}_{n,k} - 2\text{Re} \{ \xi_{l,n} \mathbf{c}_{l,n,k}^H \mathbf{w}_{n,k} \} \\ & - \omega_{l,n,k}^{(3)} \geq \chi_l, \quad \forall l \in \mathcal{S} \setminus \{n\}, \quad (19b) \end{aligned}$$

where $\mathbf{A}_{l,n,k}$, $\mathbf{b}_{n,k}$, $\mathbf{c}_{l,n,k}$, $\omega_{n,k}^{(1)}$, $\omega_{n,k}^{(2)}$, and $\omega_{l,n,k}^{(3)}$ are intermediate variables that do not related to $\mathbf{w}_{n,k}$, whose specific expressions are given in Appendix C, and the variables with bar are obtained from the previous iteration. Notice that constraint (19a) is non-convex due to $\psi_{n,k}(\mathbf{w}_{n,k})$ is a quadratic function, thus we can leverage the Taylor expansion to approximate it as

$$\begin{aligned} \psi_{n,k}(\mathbf{w}_{n,k}) & \geq \psi_{n,k}(\bar{\mathbf{w}}_{n,k}) \\ & + 4\text{Re} \{ \nabla \psi_{n,k}(\mathbf{w}_{n,k} - \bar{\mathbf{w}}_{n,k}) \} \triangleq \psi_{n,k}^{\text{low}}(\mathbf{w}_{n,k}), \quad (20) \end{aligned}$$

where $\nabla \psi_{n,k} = \left(2\text{Re} \{ \mathbf{b}_{n,k}^H \bar{\mathbf{w}}_{n,k} \} + \omega_{n,k}^{(1)} \right) \mathbf{b}_{n,k}^H$. By replacing $\psi_{n,k}(\mathbf{w}_{n,k})$ with $\psi_{n,k}^{\text{low}}(\mathbf{w}_{n,k})$ in (19a) (equivalent to (18b)), **P4** becomes a convex optimization issue that can be solved by interior-point method (IPM).

2) *Receiving Beamforming Optimization*: Similarly, we set $\mathbf{v}_n = \mathbf{0}_{M^r \times 1}$ for $n \notin \mathcal{S}$, and re-express **P3** as

$$\mathbf{P5} : \max_{\mathbf{v}_n, \chi_n \geq 0} \chi_n, \quad (21a)$$

$$\text{s.t. } f_n - \delta_n g_n \geq \chi_n, \quad (21b)$$

$$\|\mathbf{v}_n\| = 1, \quad (21c)$$

where (21b) is further derived as

$$-\mathbf{v}_n^H \mathbf{A}_n \mathbf{v}_n + \underbrace{\left[2\text{Re} \{ \mathbf{b}_n^H \mathbf{v}_n \} \right]^2}_{\triangleq \psi_n(\mathbf{v}_n)} \geq \chi_n, \quad (22)$$

where $\mathbf{A}_n = 2\delta_n \sum_{i \in \mathcal{N}} \xi_{n,i} \sum_{k,j \in \mathcal{K}_i} (c_i[k,j] \mathbf{H}_{i,k} \bar{\mathbf{w}}_{i,k} \bar{\mathbf{w}}_{i,j}^H \mathbf{H}_{i,j}^H + \delta_n \sigma^2 \mathbf{I}_{M^r})$, and $\mathbf{b}_n = \sum_{k,j \in \mathcal{K}_n} c_n[k,j] \mathbf{H}_{n,j} \bar{\mathbf{w}}_{n,j}$. Analogously, we approximate the convex term $\psi_n(\mathbf{v}_n)$ using its Taylor expansion below:

$$\psi_n(\mathbf{v}_n) \geq \psi_n(\bar{\mathbf{v}}_n) + 4\text{Re} \{ \nabla \psi_n(\mathbf{v}_n - \bar{\mathbf{v}}_n) \} \triangleq \psi_n^{\text{low}}(\mathbf{v}_n), \quad (23)$$

where $\nabla \psi_n = 2\text{Re} \{ \mathbf{b}_n^H \bar{\mathbf{v}}_n \} \mathbf{b}_n^H$. By substituting (23) into (22) (equivalent to (21b)) while omitting (21c), **P5** is convex. After solving **P5**, we normalize the attained $\mathbf{v}_n$ to a unit vector via $\mathbf{v}_n = \mathbf{v}_n / \|\mathbf{v}_n\|$, so as to satisfy (21c).

D. Overall Solution

Algorithm 1 Joint Batching and Beamforming Algorithm for P1 (JB²-Synchronous)

```

1 Input: Task set  $\mathcal{N}$ , performance requirements  $\{D_n, A_n\}$ ,
   MIMO channel  $\{\mathbf{H}_{n,k}\}$ , correlation matrix  $\{\mathbf{C}_n\}$ .
2 Initialize:  $\mathcal{S}^{(0)} = \emptyset$ ,  $\bar{\mathbf{w}}, \bar{\mathbf{v}}, I^{\max}$ .
3 Sort the tasks in  $\mathcal{N}$  in ascending order based on  $\{A_n\}$ .
4 for  $r \in \{1, 2, \dots, N\}$  do
5    $|\mathcal{S}| = r$ ,  $\mathcal{T} = \{n | n \in \mathcal{N}, (\rho_1 + \rho_2 |\mathcal{S}|) \leq D_n - T\}$ .
6   if  $|\mathcal{T}| < |\mathcal{S}|$  then
7     Batching result is  $\mathcal{S}^{(r-1)}$ .
8   else
9     Batch the first  $|\mathcal{S}|$  tasks in  $\mathcal{T}$  to obtain  $\mathcal{S}$ .
10    for  $i \in \{1, 2, \dots, I^{\max}\}$  do
11      Optimize  $\mathbf{w}_{n,k}$  for each  $n \in \mathcal{S}, k \in \mathcal{K}_n$  by solving
        P4.
12      Optimize  $\mathbf{v}_n$  for each  $n \in \mathcal{S}$  by solving P5.
13      Update  $\bar{\mathbf{w}} = \mathbf{w}, \bar{\mathbf{v}} = \mathbf{v}$ .
14    end for
15    if P3 is infeasible then
16      Batching result is  $\mathcal{S}^{(r-1)}$ .
17    else
18       $\mathcal{S}^{(r)} = \mathcal{S}$ .
19    end if
20  end if
21 end for
22 Output:  $\{\mathbf{y}, \mathbf{w}, \mathbf{v}\}$ .
```

The overall solution for **P1** is summarized in Algorithm 1 (JB²-Synchronous). In this algorithm, we sequentially construct **P2** by traversing all possible batch size and determine the batching result. Then we optimize transmission and receiving beamforming via alternatively solving **P4** and **P5**, whilst checking the feasibility of **P3**. Finally, we find the appropriate batch size as well as derive the corresponding batching and beamforming decisions. The convergence of JB²-Synchronous is guaranteed by the following proposition.

Proposition 2: The beamforming optimization converges within a finite number of iterations, and the objective value of

P1 increases monotonically until the optimal batching decision is reached.

Proof: Please refer to Appendix D. ■

The complexity of Algorithm 1 is analyzed below. Denote $I^{\max}$ as the number of alternative optimization iterations, then beamforming design has the complexity of $\mathcal{O}(I^{\max}(K+N)M^{3.5})$, where $M = \max\{M^{\text{tx}}, M^{\text{rx}}\}$. Since the loop related to r is repeated N times in the worst case, the total complexity is $\mathcal{O}(NI^{\max}(K+N)M^{3.5})$.

IV. AI-GENERATED SOLUTION FOR EDGE INFERENCE WITH ASYNCHRONOUS TASK ARRIVALS

In this part, we delve into the general asynchronous task arrival case, where the inference tasks generated by LA sensors are featured by stochastic arrival instants, and the BS should form multiple batches to satisfy their distinct completion deadlines. Unlike previous works [17], [19] that rely on precise prediction of future arrival instants, we focus on designing an AI-generated online batching policy to adapt to the dynamic and uncertain nature of task arrivals. The problem formulation and MDP modeling is first illustrated.

A. Problem Formulation and MDP Modeling

For asynchronous task arrivals, we aim to maximize the inference throughput by jointly optimizing number of batches B , batching strategy $\mathbf{y} = \{y_{n,b} : \forall n, b\}$, batch starting time $\mathbf{t} = \{t_b : \forall b\}$, and beamforming $\{\mathbf{w}, \mathbf{v}\}$. The problem is formulated as

$$\mathbf{P6} : \max_{B, \mathbf{y}, \mathbf{t}, \mathbf{w}, \mathbf{v}} \sum_{n \in \mathcal{N}} \sum_{b \in B} y_{n,b}, \quad (24a)$$

$$\text{s.t. } B \in \{1, 2, \dots, N\}, y_{n,b} \in \{0, 1\}, \forall n, b, \quad (6), (8), (9), (11), (12e) \quad (24b)$$

When an arbitrary task n is generated, we can design the beamforming $\{\mathbf{w}_{n,k}, \eta_n, \mathbf{v}_n : k \in \mathcal{K}_n\}$ according to Section III-C, thereby judging whether the task meets its inference accuracy constraint, while discarding the violated task. Note that even with given $\{\mathbf{w}, \mathbf{v}\}$, it is still non-trivial to solve the NP-hard issue **P6**, where the number of optimization variables in $\mathbf{y}$ and $\mathbf{t}$ varies with B . Additionally, we consider a practical yet challenging scenario, i.e., the arrival instant T_n and deadline D_n of each task n are not known a priori, and the resultant information uncertainty makes **P6** more intractable.⁶

Towards this end, we propose to determine $\{B, \mathbf{y}, \mathbf{t}\}$ in an online fashion. Specifically, the whole inference time period is divided into a number of slots $\zeta \in \{1, \dots, \zeta^{\max}\}$ with equal slot duration τ , then we model **P6** as an MDP below.

1) *State:* In each slot ζ , the state includes the remaining time to the completion deadline of each task n , i.e., $o_n^{\text{ddl}}(\zeta) = D_n - \zeta\tau$ (if there is no task n then $o_n^{\text{ddl}}(\zeta) = 0$), and define set $\mathcal{N}(\zeta) = \{n | n \in \mathcal{N}, o_n^{\text{ddl}}(\zeta) > 0\}$, the number of unprocessed tasks $o^{\text{num}}(\zeta) = |\mathcal{N}(\zeta)|$, and the busy time of

⁶With prior knowledge, we can invoke traditional optimization technique to solve **P6** after sophisticated mathematical transformation. This serves as a baseline in our simulation.

BS for processing the previous batch $o^{\text{bus}}(\zeta)$. Then the state is written as⁷

$$\mathbf{o}(\zeta) = \{o_n^{\text{ddl}}(\zeta), o^{\text{num}}(\zeta), o^{\text{bus}}(\zeta) : \forall n\}. \quad (25)$$

2) *Action:* The BS takes actions to balance a two-fold tradeoff. First, if the BS does not start a batch, the inference throughput potentially increases owing to more tasks can be batched, whereas the waiting latency ascends. Let $a^{\text{sta}}(\zeta) \in \{0, 1\}$ denote the first action dimension, where $a^{\text{sta}}(\zeta) = 1$ indicates batching starts, and $a^{\text{sta}}(\zeta) = 0$ means waiting. If the BS is busy with $o^{\text{bus}}(\zeta) > 0$, $a^{\text{sta}}(\zeta)$ is forced to be 0. Second, batching many tasks concurrently can enlarge the inference throughput, while raising the BS's busy time and sacrificing the ability for processing future tasks. The BS employs $a_n^{\text{bat}}(\zeta)$ to select batched tasks among $\mathcal{N}(\zeta)$, where $a_n^{\text{bat}}(\zeta) = 1$ means processing task n in the current batch, otherwise $a_n^{\text{bat}}(\zeta) = 0$. Note that $a_n^{\text{bat}}(\zeta) = 1$ only takes effect when $a^{\text{sta}}(\zeta) = 1$ and $o_n^{\text{ddl}}(\zeta) > 0$. Then we express the action as

$$\mathbf{a}(\zeta) = \{a^{\text{sta}}(\zeta), a_n^{\text{bat}}(\zeta) : \forall n\}. \quad (26)$$

Given $\mathbf{a}(\zeta)$ in all time slots, we can reconstruct $\{B, \mathbf{y}, \mathbf{t}\}$.

3) *Reward:* We design a reward function to evaluate the inference throughput whilst penalizing the violation of deadline requirements, i.e., $r(\zeta) = \sum_{n \in \mathcal{N}(\zeta)} r_n(\zeta)$ with

$$\begin{aligned} r_n(\zeta) &= \begin{cases} I_n(\zeta) - \varsigma(1 - I_n(\zeta)), & \text{if } a^{\text{sta}}(\zeta) = 1, a_n^{\text{bat}}(\zeta) = 1, \\ -\varsigma \mathbb{I}\{o_n^{\text{ddl}}(\zeta) \leq \tau\}, & \text{otherwise.} \end{cases} \end{aligned} \quad (27)$$

where $I_n(\zeta) = \mathbb{I}\{\rho_1 + \rho_2 \sum_{i \in \mathcal{N}(\zeta)} a_i^{\text{bat}}(\zeta) \leq o_n^{\text{ddl}}(\zeta)\}$ denotes whether task n is completed before its deadline, $\mathbb{I}\{x\} = 1$ if x is true, otherwise $\mathbb{I}\{x\} = 0$. If a batched task violates the latency constraint ($I_n(\zeta) = 0, a_n^{\text{bat}}(\zeta) = 1$), or an unbatched task will reach its deadline ($o_n^{\text{ddl}}(\zeta) \leq \tau, a_n^{\text{bat}}(\zeta) = 0$), we add negative penalty with ς representing the penalty parameter.

4) *State Transition:* take the transition from $\mathbf{o}(\zeta)$ to $\mathbf{o}(\zeta + 1)$ as an example. For $o_n^{\text{ddl}}(\zeta) > 0$, we have

$$\begin{aligned} o_n^{\text{ddl}}(\zeta + 1) &= \begin{cases} 0, & \text{if } a^{\text{sta}}(\zeta) = 1, a_n^{\text{bat}}(\zeta) = 1, \\ \{o_n^{\text{ddl}}(\zeta) - \tau\}^+, & \text{otherwise,} \end{cases} \end{aligned} \quad (28)$$

which implies that a task is removed when it is processed or reaches the deadline, where $\{x\}^+ = \max\{x, 0\}$. For $o_n^{\text{ddl}}(\zeta) = 0$, we set $o_n^{\text{ddl}}(\zeta + 1) = 0$ when task n has been removed or has not arrived yet; if it is a newly arriving task in slot $\zeta + 1$ with $T_n = (\zeta + 1)\tau$, then $o_n^{\text{ddl}}(\zeta + 1) = D_n - T_n$.

⁷Note that the overlapping indicator $\xi_{n,i}$ is objective and independent of the batching variables in the asynchronous case. For instance, if two tasks n and i are generated at slot ζ , we optimize the beamforming with $\xi_{n,i} = 1$. If the accuracy requirement for task n is met but task i is not with $\mathbb{E}[\|\mathbf{e}_n\|^2] \leq \gamma^2 \left(1 - \frac{A_n}{A_0}\right)$, $\mathbb{E}[\|\mathbf{e}_i\|^2] > \gamma^2 \left(1 - \frac{A_i}{A_0}\right)$, we set $o_n^{\text{ddl}}(\zeta) = D_n - T_n$ and $o_i^{\text{ddl}}(\zeta) = 0$, before executing the batching action through DRL.

$$o^{\text{bus}}(\zeta + 1) = \begin{cases} \{o^{\text{bus}}(\zeta) - \tau\}^+, & \text{if } o^{\text{bus}}(\zeta) > 0, \\ \rho_1 + \rho_2 \sum_{n \in \mathcal{N}(\zeta)} a_n^{\text{bat}}(\zeta) - \tau, & \text{if } a^{\text{sta}}(\zeta) = 1, \\ 0, & \text{otherwise.} \end{cases} \quad (29)$$

B. Preliminaries of Diffusion Model

To be specific, diffusion model aims to infer the optimized action distribution $\mathbf{x}_0(\zeta)$ under environment state $\mathbf{o}(\zeta)$, denoted by $\mathbf{a}(\zeta) \sim \mathbf{x}_0(\zeta) = \pi_{\theta}(\mathbf{o}(\zeta))$, where θ is the parameters of diffusion model, and π indicates the policy. To achieve this, two critical processes are comprised, in which the forward process adds noise to $\mathbf{x}_0(\zeta)$ for U steps until an indistinguishable standard Gaussian noise $\mathbf{x}_U(\zeta)$ is attained; while the reverse process gradually denoises $\mathbf{x}_U(\zeta)$ to recover $\mathbf{x}_0(\zeta)$, and leverages a DNN with parameter θ to predict the removed noise $\varepsilon_{\theta}(\mathbf{x}_u(\zeta), u, \mathbf{o}(\zeta))$ at each denoising step u . Note that state $\mathbf{o}(\zeta)$ serves as the conditioning information for guiding the reverse process, enabling adaptive decision-making in dynamic environment. Mathematically, the forward and reverse processes at adjacent steps $u - 1$ to u can be respectively characterized by [33]

$$\mathbf{x}_{u-1}(\zeta) = \mu_{\theta}(\mathbf{x}_u(\zeta), u, \mathbf{o}(\zeta)) + \sqrt{\bar{\Gamma}_u} \bar{\epsilon}, \quad (31)$$

$$\begin{aligned} \mu_{\boldsymbol{\theta}}(\mathbf{x}_u(\zeta), u, \mathbf{o}(\zeta)) = & \frac{1}{\sqrt{\Lambda_u}} \left[\mathbf{x}_u(\zeta) \right. \\ & \left. - \frac{\Gamma_u}{\sqrt{1 - \Lambda_u}} \boldsymbol{\varepsilon}_{\boldsymbol{\theta}}(\mathbf{x}_u(\zeta), u, \mathbf{o}(\zeta)) \right], \end{aligned} \quad (32)$$

The diagram illustrates the framework of the Diffusion-based Actor-Critic. It shows the interaction between the Environment, the Target actor network, the Diffusion-based actor network, and the Critic/Target critic networks.

- Environment:** Provides state $\mathbf{s}_t$ and action $\mathbf{a}_t$ to the Diffusion-based actor network. It also receives the action $\mathbf{a}_t$ from the Diffusion-based actor network and provides the reward r_t to the Replay buffer.
- Target actor network $\pi_{\hat{\theta}}$:** Provides a target action distribution $\Pr\{a(\zeta) = i\}$ to the Diffusion-based actor network.
- Diffusion-based actor network π_{θ} :**
 - Forward process:** Generates a sequence of actions $\mathbf{x}_1(\zeta), \dots, \mathbf{x}_N(\zeta)$ from Gaussian Noise $\mathbf{x}_1(\zeta)$ using Eq. (30).
 - Reverse process:** Generates a sequence of actions $\mathbf{x}_1(\zeta), \dots, \mathbf{x}_N(\zeta)$ from the action distribution $\mathbf{a}(\zeta)$ using Eq. (31).
 - Diffusion process:** The forward process is a sequence of denoising steps, each involving a DNN $\mathcal{D}_{\theta}$ and a noise variable $\mathbf{u}_t$.
- Critic network $\mathcal{Q}_{\phi}$ and Target critic $\hat{\mathcal{Q}}_{\phi}$:**
 - Receive the action distribution $\mathbf{a}(\zeta)$ from the Diffusion-based actor network.
 - Calculate the loss function $L_{\pi}(\theta)$ and the loss function $L_{\mathcal{Q}}(\phi)$.
 - Update their parameters θ and ϕ using the gradients.

Figure 10 is a line graph showing Inference accuracy (Y-axis, ranging from 0.4 to 1.0) versus Feature aggregation error (dB) (X-axis, ranging from -2 to 8). The graph compares four models: ModelNet 10-Shaded (blue circles), ModelNet 10-Depth (red diamonds), ModelNet 40-Shaded (yellow squares), and ModelNet 40-Depth (purple triangles). The shaded models generally maintain higher inference accuracy than the depth-based models across the range of feature aggregation errors. The 40-Depth model shows the most significant drop in accuracy as the error increases, falling below 0.4 accuracy at 8 dB.

Feature aggregation error (dB)	ModelNet 10-Shaded	ModelNet 10-Depth	ModelNet 40-Shaded	ModelNet 40-Depth
-1	0.97	0.91	0.90	0.86
0	0.97	0.91	0.90	0.86
1	0.97	0.91	0.88	0.85
2	0.96	0.91	0.87	0.84
3	0.95	0.91	0.83	0.78
4	0.93	0.89	0.78	0.68
5	0.89	0.85	0.68	0.58
6	0.82	0.78	0.58	0.48
7	0.74	0.70	0.48	0.38
8	0.65	0.58	0.45	0.32

C. Training Framework Design

As shown in Fig. 3, the training framework incorporates a diffusion-based actor network π_θ to generate actions, and a critic network Q_φ parametrized by φ to evaluate actions. In addition, two target networks denoted by $\hat{\pi}_\theta$ and $\hat{Q}_\varphi$ are introduced to prevent training oscillation.

$$L_{\pi}(\boldsymbol{\theta}) = -\mathbb{E}[Q_{\varphi}(\mathbf{o}(\zeta), \pi_{\boldsymbol{\theta}}(\mathbf{o}(\zeta)))]. \quad (33)$$

To calculate $L_\pi(\theta)$, we execute the diffusion-based actor network to output action $\mathbf{a}(\zeta)$ following the reverse process in (31), during which the DNN with parameter θ generates denoising noise in each step conditioned on state $\mathbf{o}(\zeta)$, as in (32). Subsequently, $\mathbf{o}(\zeta)$ and $\mathbf{a}(\zeta)$ are input into the critic network for yielding evaluation value Q_φ . We then compute the gradient of $L_\pi(\theta)$ with respect to θ , and update the actor network as $\theta \leftarrow \theta + lr \mathbb{E} [\nabla_{\theta} \pi(\mathbf{o}(\zeta)) \cdot \nabla_{\mathbf{a}(\zeta)} Q_\varphi(\mathbf{o}(\zeta), \mathbf{a}(\zeta))]$, where lr

denotes the learning rate. Since the critic network aims to minimize the error between the predicted and actual accumulative rewards, its loss is

$$L_Q(\varphi) = \mathbb{E}[Q_\varphi(\mathbf{o}(\zeta), \mathbf{a}(\zeta)) - Q^{\text{tar}}(\zeta)]^2, \quad (34)$$

where $Q^{\text{tar}}(\zeta) = r(\zeta) + \omega \hat{Q}\hat{\varphi}(\mathbf{o}(\zeta+1), \hat{\pi}_{\hat{\theta}}(\mathbf{o}(\zeta+1)))$, and ω denotes the discount factor. Moreover, target network parameters are updated through tardily following the actor and critic networks, i.e.,

$$\begin{aligned} \hat{\theta} &\leftarrow \kappa\theta + (1-\kappa)\hat{\theta}, \\ \hat{\varphi} &\leftarrow \kappa\varphi + (1-\kappa)\hat{\varphi}, \end{aligned} \quad (35)$$

where $\kappa \ll 1$ denotes the update rate.

D. Overall Solution

Algorithm 2 Diffusion Model-Based AI-Generated Solution Algorithm for P6 (GAI-Asynchronous)

- 1 **Training Stage:**
- 2 **Initialize:** DNN parameters of diffusion-based actor network and critic network $\theta, \varphi, \hat{\theta}, \hat{\varphi}$, replay buffer.
- 3 **for** each episode **do**
- 4 **for** $\zeta \in \{1, \dots, \zeta^{\max}\}$ **do**
- 5 Initialize a Gaussian random variable $\mathbf{x}_U(\zeta)$.
- 6 Generate $\mathbf{x}_0(\zeta)$ based on the reverse process in (31).
- 7 Execute action $\mathbf{a}(\zeta) \sim \mathbf{x}_0(\zeta) = \pi_\theta(\mathbf{o}(\zeta))$.
- 8 Calculate reward $r(\zeta)$ and transit to $\mathbf{o}(\zeta+1)$.
- 9 Store the experience in replay buffer.
- 10 **end for**
- 11 Sample experiences from the replay buffer.
- 12 Update the diffusion-based actor network by minimizing (33).
- 13 Update the critic network by minimizing (34).
- 14 Update the target networks using (35).
- 15 **end for**
- 16 **Implementation stage:**
- 17 Optimize beamforming $\{\mathbf{w}, \mathbf{v}\}$ according to Section III-C.
- 18 **for** $\zeta \in \{1, \dots, \zeta^{\max}\}$ **do**
- 19 LA sensors transmit features and form state $\mathbf{o}(\zeta)$.
- 20 Generate batching decision $\mathbf{a}(\zeta)$ using the trained diffusion-based actor network.
- 21 **end for**
- 22 **Output:** $\{B, \mathbf{y}, \mathbf{t}, \mathbf{w}, \mathbf{v}\}$.

Algorithm 2 (GAI-Asynchronous) outlines our AI-generated solution to solve **P6** for asynchronous task arrivals. During the training stage, the algorithm leverages the diffusion-based actor network to output action under each environment state, then it calculates reward and performs state transition. By sampling experiences from the replay buffer, we update actor and critic networks as well as two target networks. In the implementation stage, after optimizing the beamforming, we invoke the well-trained actor network to generate batching decisions in response to real-time task arrivals. Given the challenge of theoretically proving the convergence performance of DRL, particularly for the DRL enhanced by the diffusion model [23], [24], [25], we will instead verify the

TABLE II
SIMULATION PARAMETERS

Parameter	Value	Parameter	Value
K_n	12	Q	12544
M^{tx}	4	M^{rx}	16
σ^2	-80 dBm	v	-30 dB
α	3.8	ρ_1, ρ_2	1.502, 0.3051 ms
A_0	0.95	γ	$\sqrt{3Q}$
A_n	[0.8, 0.9]	P	1 W
$\zeta^{\max}$	200	τ	0.5 ms
U	5	Learning rate of actor and critic	$1 \times 10^{-3}, 1 \times 10^{-4}$
ς	2	Γ_u	Controlled by variational posterior scheduler [33]
ω	0.99	κ	0.005

TABLE III
COMPARISON OF ALGORITHM IMPLEMENTATION TIME

Methods	Number of inference tasks		
	$N = 30$	$N = 40$	$N = 50$
Proposed ($U = 3$)	0.466 s	0.557 s	0.608 s
Proposed ($U = 5$)	0.744 s	0.792 s	0.802 s
Proposed ($U = 7$)	1.083 s	1.136 s	1.336 s
SAC/TD3	0.239 s	0.306 s	0.343 s
MINLP-Prior	47.79 s	80.01 s	174.32 s

convergence through numerical experiments, as shown in Fig. 8. Considering that the diffusion model loads DNN for U denoising steps, the implementation complexity is given by $\mathcal{O}(I^{\max}(K+N)M^{3.5} + \zeta^{\max}UN^2)$. This presents the computational load that our approach puts into the BS.

V. PERFORMANCE EVALUATION

This part conducts comprehensively simulations to evaluate the performance of our proposed JB²-Synchronous and GAI-Asynchronous algorithms. In the simulation scenario, the BS is located at the center of a circle with radius 100 m, and all LA sensors are randomly distributed in the circle. To execute multi-task edge inference, we adopt ModelNet dataset⁸ to train a multi-view convolutional neural network (MVCNN) [35]. Specifically, ModelNet captures 12 images for each object sample, and its subsets ModelNet 40 and ModelNet 10 contain 40 and 10 popular object classes, respectively. Besides, the images in ModelNet can be rendered in shaded and depth styles, resulting in different combinations, e.g., ModelNet 40-Shaded and ModelNet 10-Depth. The MVCNN splits its VGG16 backbone before the linear classifier, and the preceding layers are deployed at each sensor for extracting feature with dimension $512 \times 7 \times 7$. Afterwards, 12 feature vectors are aggregated into a global feature vector using average pooling, which is then fed into the edge classifier. Our subsequent results are derived based on the well-trained MVCNN, and the classification of different object samples corresponds to different inference task. Other default simulation parameters⁹ are listed in Table II [19], [23], [26].

⁸The choice of ModelNet is due to its status as a classical open-source dataset for multi-view intelligent sensing, as well as its inclusion of the aircraft class, which contains over 700 samples of common LA objects.

⁹In our parameter settings, A_0 is estimated as the test accuracy of the well-trained MVCNN on ModelNet 10-Shaded dataset. For γ , we introduce Gaussian noises with varying variances into the global feature $\mathbf{f}_n$, and γ^2 is set as the largest variance that does not degrade the test accuracy.

A. Results for Synchronous Task Arrivals and Single Batch

In this subsection, we provide numerical results for synchronous task arrivals and single batch case, where the default number of inference tasks N is 20, and the latency requirements $D_n - T$ are randomly sampled from [5, 20] ms. To investigate the impact of the aforementioned spatial correlation, the feature correlation matrix is approximated employing the following manners:

1) *Approximation 1*: We calculate $\mathbf{C}_n = \frac{1}{2Q} \sum_{q=1}^{2Q} \mathbf{z}_{n,q} \mathbf{z}_{n,q}^T$ based on the knowledge of sensors' feature vectors. However, this is impractical since the knowledge cannot be obtained before feature transmission. Thus, Approximation 1 provides a performance baseline.

2) *Approximation 2*: $\mathbf{C}_n = \epsilon \mathbf{1}_{K_n \times K_n} + (1 - \epsilon) \mathbf{I}_{M^{\text{rx}}}$, where ϵ represents an empirical value to estimate the degree of correlation.

Additionally, the developed JB²-Synchronous algorithm is compared with benchmark methods below

1) *Spatial Correlation-Unawareness* [11], [13]: This method ignores the potential correlation among feature vectors from the same inference task, and sets $\mathbf{C}_n = \mathbf{I}_{M^{\text{rx}}}$ to optimize beamforming. Hence, it can be regarded as a special case of Approximation 2 with $\epsilon = 0$.

2) *Zero-Forcing (ZF)-Based Power Control* [10]: This method is widely used in AirComp to minimize the computation distortion, in which the phases of transmission beamforming are determined by ZF precoding, and the power control can be optimized by convex programming. Moreover, the receiving beamforming and batching are designed via our approach.

3) *Static Batching* [36]: The BS selects a fixed batch size in advance, and it starts parallel inference when the number of batched tasks reaches the batch size. A task is discarded if the completion time exceeds its deadline. We search an appropriate batch size catering for all possible N considered in the simulations.

4) *Sequential Inference* [37]: The BS carries out edge inference without batching capability. Therefore, all the tasks are processed in a sequential fashion according to their arrival order, and the inference latency of each task is $l_n = \rho_1 + \rho_2, \forall n$.

Fig. 4 depicts the relation between inference accuracy and feature aggregation error. As the error becomes large, the global feature $\hat{\mathbf{f}}_n$ attained by over-the-air aggregation deviates more significantly from the error-free version $\mathbf{f}_n$, reducing the inference accuracy of edge AI model. Furthermore, the MVCNN model can achieve higher inference accuracy on ModelNet 10 and shaded images, as compared to ModelNet 40 and depth images. This is because the discrimination of 10 object classes is easier than 40 classes, while the shaded images offer additional information related to shadow and texture. Particularly, this figure confirms that the feature aggregation error can be utilized as a tractable accuracy performance metric to facilitate beamforming design.

In Fig. 5, we assess the feature aggregation error versus the number of receiving antennas at the BS M^{rx} . It can be observed that the growth of M^{rx} reduces the feature aggregation error, since more antennas can provide higher spatial multiplexing gains to suppress the misalignment error as well as inter-task interference. In addition, we find that

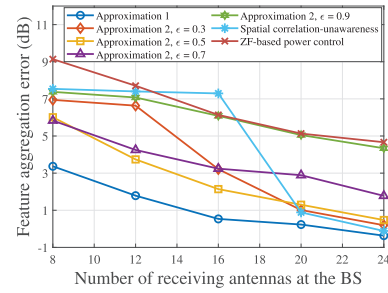


Fig. 5. Feature aggregation error versus M^{rx} .

the performance of Approximation 2 with $\epsilon = 0.5$ is close to Approximation 1, and the increase or decrease of ϵ diminishes the performance, implying that $\epsilon = 0.5$ is a reasonable estimation of spatial correlation degree. In the remainder of the simulations, the proposed approach adopts Approximation 2 with $\epsilon = 0.5$ to optimize beamforming.

For *spatial correlation-unawareness*, an interesting phenomenon is that it suffers from serious performance degradation when $M^{\text{rx}} \leq 16$, whereas the feature aggregation error significantly declines when M^{rx} further ascends. The rationale is that when the BS is equipped with a large receiving array, the symbols transmitted by different LA sensors can be distinguished exploiting the sufficient spatial degree of freedom (DoF). In this case, the influence of overlooking the feature correlation in beamforming design is unremarkable. This result reveals that the selection of ϵ should be coordinated with receiving array size in practical systems.

Fig. 6 showcases the comparison of our proposed JB²-Synchronous and benchmark methods, in which the normalized inference throughput is given by $\frac{1}{N} \sum_{n \in \mathcal{N}} \sum_{b \in \mathcal{B}} y_{n,b}$. Fig. 6 (a) displays that the normalized inference throughput monotonically descends as the number of tasks becomes large. This is owing to the inference latency grows linearly with the batch size, and inter-task interference exacerbates the aggregation error, hence only finite tasks can be batched to meet their deadline and accuracy requirements. Furthermore, the proposed algorithm outperforms four benchmark methods by 29.89%, 24.85%, 22.44%, and 49.61% in enhancing the inference throughput, respectively. Since *spatial correlation-unawareness* neglects the feature correlation, and *ZF-based power control* directly minimizes computation distortion instead of our analytical feature aggregation error in (15), and the resultant beamforming is not conducive to guaranteeing inference accuracy. For *static batching*, fixing batch size for all N restricts the improvement of inference throughput, thus its performance is lower than the proposed algorithm with adaptive batch size and task selection design. *Sequential inference* performs worse due to it prolongs the waiting time for task processing, and the inference latency is also high thanks to the omission of batching.

In Fig. 6 (b), we plot the normalized inference throughput versus average latency requirement. One can see that all curves ascend as the latency requirement relaxes. This can be explained by that the BS has more time to execute edge inference, increasing the number of tasks that can be batched. Moreover, the inference throughputs obtained by *spatial correlation-unawareness* and *ZF-based power control* are bounded when loosening the latency requirement, since the accuracy requirement becomes the bottleneck of

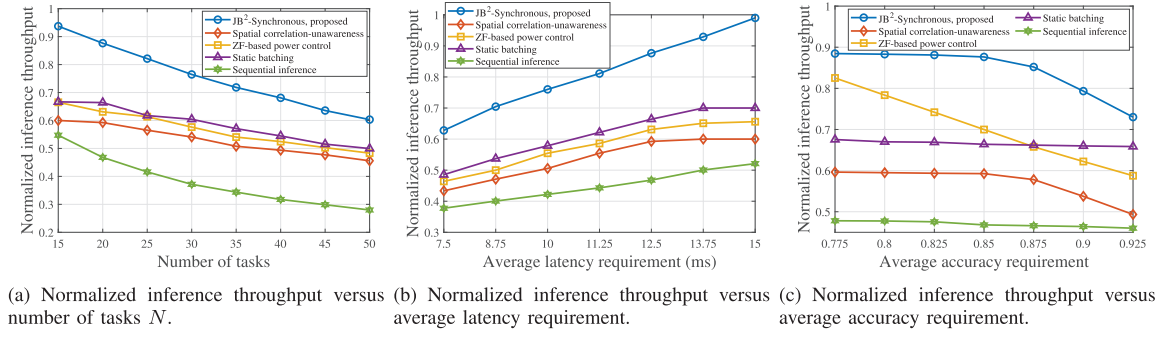


Fig. 6. Inference throughput for synchronous task arrivals.

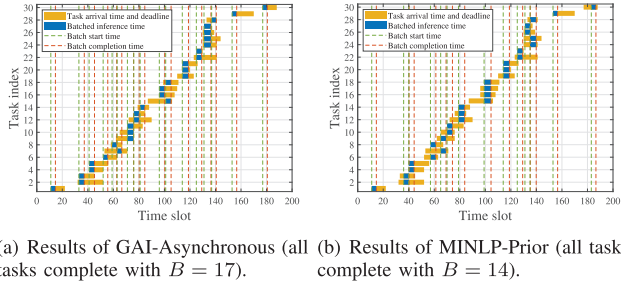


Fig. 7. Illustration of batching strategy for asynchronous task arrivals.

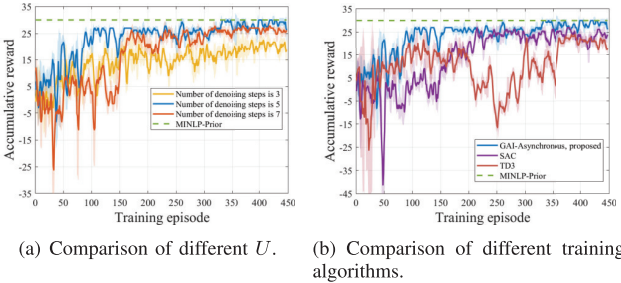


Fig. 8. Convergence performance evaluation.

task completion. Numerical results indicate that the proposed JB²-Synchronous achieves outstanding performance, i.e., the normalized inference throughput is 34.08%, 29.07%, 24.79%, and 45.03% higher than benchmark methods.

Fig. 6 (c) validates the performance of the proposed approach under varying average accuracy requirements. It is seen that the inference throughput of JB²-Synchronous remains stable initially and then decreases as the accuracy requirement tightens. The rationale is that stricter accuracy constraints impose more stringent limits on feature aggregation errors, reducing the number of completed tasks under finite spatial DoFs. Consistent with the aforementioned results, we can further observe the superiority of JB²-Synchronous over baselines.

B. Results for Asynchronous Task Arrivals

This subsection presents results for asynchronous task arrivals, where $N = 30$ tasks arrive randomly in time period $[0, 100]$ ms (which is divided into 200 slots with slot duration 0.5 ms), and latency requirements $D_n - T_n$ are within $[2, 10]$ ms. The DNN of the diffusion model possesses two hidden layers with 256 neurons, and the input includes Sinusoidal position embedding to encode denoising steps. In addition

to the above-mentioned benchmark methods, the following schemes are compared to demonstrate the effectiveness of our proposed GAI-Asynchronous algorithm:

1) *Mixed Integer Nonlinear Programming (MINLP)-Prior* [19]: This method assumes that task arrival time $T_n, \forall n$ and deadline $D_n, \forall n$ are known a priori, such that **P6** can be transformed into a series of MINLP issues, as detailed in Appendix E. Since this method employs prior information that is difficult to acquire in practice, it provides a performance baseline for our online policy.

2) *Soft Actor-Critic (SAC)* [29]: We invoke SAC as a DRL benchmark for comparing the learning efficiency. This method uses fully-connected DNN to yield action distribution, and promotes exploration by reaping the benefit of maximum entropy objective.

3) *Twin Delayed Deep Deterministic Policy Gradient (TD3)*: TD3 is another DRL benchmark, which maps each state to a deterministic action, and applies some tricks to facilitate training. Its actor network is also a fully-connected DNN.

The batching strategy obtained by the proposed algorithm and MINLP-Prior is illustrated in Fig. 7, where the yellow region shows the arrival time and deadline of each task, and the blue region portrays the batched inference time. If the blue region of a task is fully covered by the yellow region, the task is completed before the deadline. Moreover, if the blue regions of several tasks are aligned, these tasks are assembled into the same batch. One can observe that both GAI-Asynchronous and MINLP-Prior complete all the 30 inference tasks, indicating that our approach can effectively learn the online batching strategy, and achieve comparable inference throughput compared to the benchmark with prior task information. Additionally, owing to the uncertainty of future task arrivals, GAI-Asynchronous tends to start batched inference as early as possible. Another important observation is that MINLP-Prior completes all tasks with few batches ($B = 14$) than GAI-Asynchronous ($B = 17$). The reason is that in online implementation without prior information, our proposed approach should appropriately control the waiting latency and the BS's busy time, thereby ensuring the ability for processing future tasks.

Fig. 8 shows the training convergence of the proposed GAI-Asynchronous. It can be seen that the accumulative reward curves ascend over the training episode, and the final performance is close to the upper bound given by MINLP-Prior. This is because the proposed algorithm learns from the

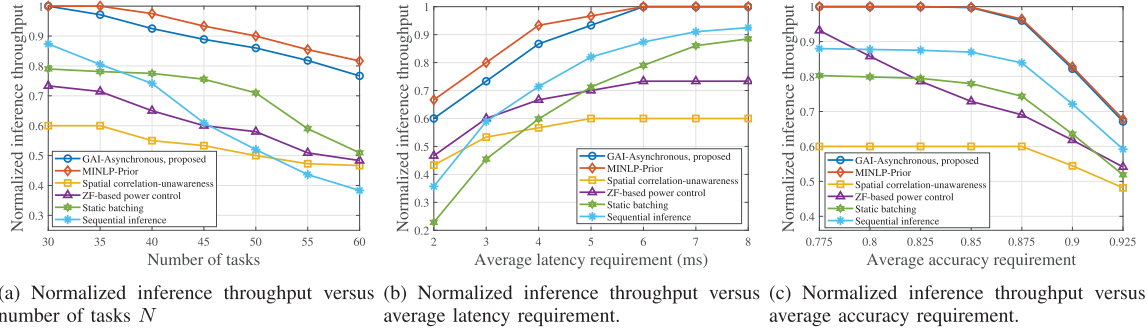


Fig. 9. Inference throughput for asynchronous task arrivals.

environmental rewards, and continuously improves the policy yielded by diffusion-based actor network. More specifically, Fig. 8 (a) studies the impact of denoising steps on the training convergence, and we find that the accumulative reward first increases then decreases as the number of denoising steps U ascends. On the one hand, enlarging U means that the action can be refined with more steps in the generation process, stabilizing network outputs and reducing training oscillations. On the other hand, when U becomes very large, the diffusion model loses its exploration ability because of the excessing denoising, degrading the convergence performance in complicated environment. As a result, we choose $U = 5$ to strike a good balance.

Fig. 8 (b) compares the training procedure of GAI-Asynchronous and DRL benchmarks. We observe that the proposed approach achieves superior performance, and the accumulative reward is 38.63% and 56.43% higher than SAC and TD3, respectively. This performance improvement is attributed to the diffusion-based actor network, which utilizes multiple denoising steps to generate the precise mapping from state to action spaces, outperforming the fully-connected DNN used by DRL benchmarks. This result unveils that the relationship between diffusion model and DRL are complementary and mutually beneficial, providing critical opportunities for developing high-performance decision-making approaches.

Thereafter, we provide the algorithm implementation time in Table II. As can be observed, all the learning-based methods load the trained actor network, and consume about 1 second to make real-time batching actions. Meanwhile, the increment of the time consumption is negligible when the number of tasks grows. Nevertheless, MINLP-Prior requires to iteratively solve the MINLP issues to acquire the offline decisions, resulting in a much higher implementation time than our proposal. Besides, with the increase of U , the time consumption of GAI-Asynchronous also ascends since the DNN for denoising are executed more times. SAC and TD3 only need to execute the actor networks once, hence they have the lowest implementation time.

Fig. 9 delineates the inference throughput performance for asynchronous task arrivals. Similar to the discussion of Fig. 6, the normalized inference throughput decreases with task amount as well as average accuracy requirement, and increases with average latency requirement. Furthermore, it is seen that the proposed GAI-Asynchronous achieves comparable performance to MINLP-Prior, and outperforms other benchmark methods. Numerical results demonstrate that the normalized

inference throughput of our algorithm is only 4% lower than MINLP-Prior, and 40.25%, 31.46%, 21.18%, and 29.88% higher than *spatial correlation-unawareness*, *ZF-based power control*, *static batching*, *sequential inference*, respectively.

C. Validation of Algorithm Extensibility

Fig. 10 demonstrates the effectiveness of the proposed approach in an advanced real-world drone detection task. In this experiment, we fine-tune a pre-trained YOLOv8 model using the Drone Dataset [38]. The first convolutional layer is deployed on LA sensors for feature extraction, while the remaining components of YOLOv8 are deployed at the BS to perform detection. The detection results show a strong correspondence with the ground truth labels, i.e., all drones are accurately detected with correct positions. In contrast, the benchmark *ZF-based power control* method exhibits instances of missed or falsely detected drones. This improvement is attributed to the proper design of MIMO beamforming in our approach, which effectively mitigates feature aggregation errors caused by over-the-air transmission, thereby maintaining the model's detection performance.

In Fig. 11, we extend the proposed approach to the Rician channel model. The MIMO channel between sensor $\langle n, k \rangle$ and BS is $\mathbf{H}_{n,k} = \sqrt{v d_{n,k}^{-\alpha}} \left(\sqrt{\frac{K_f}{1+K_f}} \mathbf{H}_{n,k}^{\text{LoS}} + \sqrt{\frac{1}{1+K_f}} \tilde{\mathbf{H}}_{n,k} \right)$, where $\mathbf{H}_{n,k}^{\text{LoS}}$ and $\tilde{\mathbf{H}}_{n,k}$ are the line-of-sight (LoS) component and NLoS component, respectively, and K_f denotes the Rician factor. We can see that the feature aggregation error initially decreases and then stabilizes as K_f increases. This is due to that the enhanced LoS propagation is beneficial for feature uploading, whereas further reduction in feature aggregation error is constrained by inter-task interference. Additionally, the proposed approach (adopting Approximation 2 with $\epsilon = 0.5$) demonstrates significant performance improvement over baseline beamforming methods, confirming the extensibility of our approach for different channel models.

Fig. 12 examines the impact of $\zeta^{\max}$ on the performance of GAI-asynchronous, where time period $[0, 100]$ ms is divided into various number of time slots, with slot duration τ determined accordingly. We observe that the inference throughput ascends as $\zeta^{\max}$ grows, though the rate of increase diminishes once $\zeta^{\max}$ reaches a sufficiently large value, i.e., 200. A larger $\zeta^{\max}$ allows the agent to make more accurate batching decisions over shorter time intervals. However, extremely short action intervals are unnecessary, since the inference throughput is bounded by delay and accuracy constraints. Given that the algorithm implementation time scales approximately linearly



Fig. 10. Performance on drone detection task using YOLOv8 model and Drone Dataset.

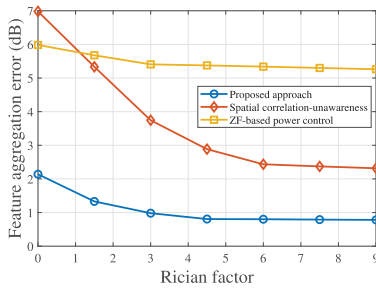


Fig. 11. Extension to Rician channel model.

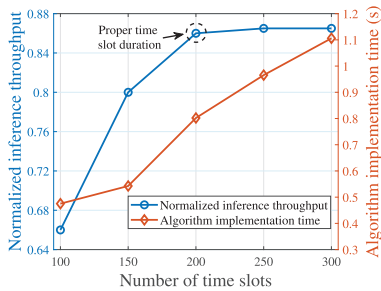


Fig. 12. Impact of various number of time slots $\zeta^{\max}$.

with $\zeta^{\max}$, it is proper to choose a slot duration near the turning point where the throughput gain starts to level off.

VI. CONCLUSION

In this work, we proposed a multi-task over-the-air edge inference system to achieve high-precision and intelligent sensing for LA airspace. To unleash the full potential of this system, batching and beamforming were jointly optimized to maximize the inference throughput, satisfying the completion deadline and inference accuracy constraints. Since the inference tasks possess stochastic arrival instants and distinct completion requirements, our solution sequentially handled two cases. In the case with synchronous task arrivals and single batch, JB²-Synchronous algorithm was developed to design transceiver beamforming by considering the spatial correlation among sensors, whilst batching was optimized by finding the maximum feasible batch size. In the synchronous task arrival case, we proposed the GAI-Asynchronous algorithm, which innovatively incorporated a diffusion-based actor network to generate batching actions in an online manner, judiciously balancing the completed task amount, waiting latency, and

BS's busy time. Through numerical results, we showcased the importance of capturing the feature correlation among sensors, especially when the spatial DoF is insufficient. Compared to offline batching optimization with prior task information, our online policy achieved comparable performance with higher implementation efficiency. Moreover, the proposed algorithms outperformed benchmark methods in terms of inference throughput and training convergence. Future work will delve into more complex LA applications, such as video analysis, and develop sophisticated beamforming designs by exploring both spatial and dynamic temporal correlations.

REFERENCES

- [1] Y. Fu, P. Qin, J. Zhang, and Z. Lu, "Joint AI inference and target tracking at network edge: A hybrid offline-online design for UAV-enabled network," *IEEE Trans. Wireless Commun.*, vol. 23, no. 12, pp. 17959–17973, Dec. 2024.
- [2] H.-J. Moon and C.-B. Chae, "Cooperative ground-satellite scheduling and power allocation for urban air mobility networks," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 1, pp. 218–233, Jan. 2025.
- [3] Federal Aviation Admin. (Apr. 2024). *FAA Aerospace Forecast Fiscal Years 2024–2044*. [Online]. Available: <https://www.faa.gov/dataresearch/aviation/aerospaceforecasts/faa-aerospace-forecast-fy-2024-2044>
- [4] B.-H. Liao, C.-Y. Lee, and L.-C. Wang, "Aerodynamics-based collision-free control of connected drones in complex urban low-altitude airspace using distributional reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 73, no. 7, pp. 9763–9775, Jul. 2024.
- [5] ITU-R. (2023). *Framework and Overall Objectives of the Future Development of IMT for 2030 and Beyond*. [Online]. Available: <https://www.itu.int/en/ITU-R/study-groups/rsg5/rwp5d/imt-2030/Pages/default.aspx>
- [6] F. Liu et al., "Integrated sensing and communications: Toward dual-functional wireless networks for 6G and beyond," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 6, pp. 1728–1767, Jun. 2022.
- [7] S. Duan et al., "Distributed artificial intelligence empowered by end-edge-cloud computing: A survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 591–624, 1st Quart., 2023.
- [8] J. Shao, Y. Mao, and J. Zhang, "Task-oriented communication for multi-device cooperative edge inference," *IEEE Trans. Wireless Commun.*, vol. 22, no. 1, pp. 73–87, Jan. 2023.
- [9] P. R. Sutradhar et al., "Look-up-table based processing-in-memory architecture with programmable precision-scaling for deep learning applications," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 2, pp. 263–275, Feb. 2022.
- [10] X. Shi, J. Du, J. Wang, K. Huang, and T. Q. S. Quek, "Beamforming design for massive MIMO-aided over-the-air computation: A mutual information perspective," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 14335–14349, Oct. 2024.
- [11] X. Chen, K. B. Letaief, and K. Huang, "On the view-and-channel aggregation gain in integrated sensing and edge AI," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 9, pp. 2292–2305, Sep. 2024.
- [12] Z. Liu, Q. Lan, A. E. Kalør, P. Popovski, and K. Huang, "Over-the-Air multi-view pooling for distributed sensing," *IEEE Trans. Wireless Commun.*, vol. 23, no. 7, pp. 7652–7667, Jul. 2024.

- [13] Z. Zhuang, D. Wen, Y. Shi, G. Zhu, S. Wu, and D. Niyato, "Integrated sensing-communication-computation for over-the-air edge AI inference," *IEEE Trans. Wireless Commun.*, vol. 23, no. 4, pp. 3205–3220, Apr. 2024.
- [14] P. Zhang, D. Wen, G. Zhu, Q. Chen, K. Han, and Y. Shi, "Collaborative edge AI inference over cloud-RAN," *IEEE Trans. Commun.*, vol. 72, no. 9, pp. 5641–5656, Sep. 2024.
- [15] NVIDIA.(2022). *NVIDIA TensorRT Documentation*. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/>
- [16] W. Cui, Q. Chen, H. Zhao, M. Wei, X. Tang, and M. Guo, "E2Bird: Enhanced elastic batch for improving responsiveness and throughput of deep learning services," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 6, pp. 1307–1321, Jun. 2021.
- [17] W. Shi, S. Zhou, Z. Niu, M. Jiang, and L. Geng, "Multiuser co-inference with batch processing capable edge server," *IEEE Trans. Wireless Commun.*, vol. 22, no. 1, pp. 286–300, Jan. 2023.
- [18] Z. Liu, Q. Lan, and K. Huang, "Resource allocation for multiuser edge inference with batching and early exiting," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 4, pp. 1186–1200, Apr. 2023.
- [19] Y. Cang, M. Chen, and K. Huang, "Joint batching and scheduling for high-throughput multiuser edge AI with asynchronous task arrivals," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 13782–13795, Oct. 2024.
- [20] M. Xu et al., "Unleashing the power of edge-cloud generative AI in mobile networks: A survey of AIGC services," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 2, pp. 1127–1170, 2nd Quart., 2024.
- [21] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," in *Proc. ICML*, 2015, pp. 2256–2265.
- [22] S. Krishnamoorthy, S. M. Mashkaria, and A. Grover, "Diffusion models for black-box optimization," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2023, pp. 17842–17857.
- [23] H. Du et al., "Diffusion-based reinforcement learning for edge-enabled AI-generated content services," *IEEE Trans. Mobile Comput.*, vol. 23, no. 9, pp. 8902–8918, Sep. 2024.
- [24] B. Du et al., "YOLO-based semantic communication with generative AI-aided resource allocation for digital twins construction," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 7664–7678, Mar. 2024.
- [25] Z. Liu et al., "DNN partitioning, task offloading, and resource allocation in dynamic vehicular networks: A Lyapunov-guided diffusion-based reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 24, no. 3, pp. 1945–1962, Mar. 2025.
- [26] C. Zhong, H. Yang, and X. Yuan, "Over-the-air federated multi-task learning over MIMO multiple access channels," *IEEE Trans. Wireless Commun.*, vol. 22, no. 6, pp. 3853–3868, Jun. 2023.
- [27] J. Sokolic, R. Giryes, G. Sapiro, and M. R. D. Rodrigues, "Robust large margin deep neural networks," *IEEE Trans. Signal Process.*, vol. 65, no. 16, pp. 4265–4280, Aug. 2017.
- [28] Y. Chen, Z. Chang, G. Min, S. Mao, and T. Hämmäläinen, "Joint optimization of sensing and computation for status update in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 22, no. 11, pp. 8230–8243, Nov. 2023.
- [29] P. Qin, Y. Fu, Z. Yu, J. Zhang, and X. Zhao, "URLLC-aware trajectory plan and beamforming design for NOMA-aided UAV integrated sensing, communication, and computation networks," *IEEE Trans. Veh. Technol.*, vol. 74, no. 1, pp. 1610–1625, Jan. 2025.
- [30] P. Qin, Y. Fu, R. Ding, and H. He, "Competition-awareness partial task offloading and UAV deployment for multitier parallel computational Internet of Vehicles," *IEEE Syst. J.*, vol. 18, no. 3, pp. 1753–1764, Sep. 2024.
- [31] P. Qin, Y. Fu, Y. Xie, K. Wu, X. Zhang, and X. Zhao, "Multi-agent learning-based optimal task offloading and UAV trajectory planning for AGIN-power IoT," *IEEE Trans. Commun.*, vol. 71, no. 7, pp. 4005–4017, Jul. 2023.
- [32] H. Du et al., "Enhancing deep reinforcement learning: A tutorial on generative diffusion models in network optimization," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 4, pp. 2611–2646, 4th Quart., 2024.
- [33] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 6840–6851.
- [34] P. Qin, M. Fu, Y. Fu, R. Ding, and X. Zhao, "Collaborative edge computing and program caching with routing plan in C-NOMA-Enabled space-air-ground network," *IEEE Trans. Wireless Commun.*, vol. 23, no. 12, pp. 18302–18315, Dec. 2024.
- [35] J.-C. Su, M. Gadelha, R. Wang, and S. Maji, "A deeper look at 3D shape classifiers," 2018, *arXiv:1809.02560*.
- [36] A. Ali, R. Pincirol, F. Yan, and E. Smirni, "BATCH: Machine learning inference serving on serverless platforms with adaptive batching," in *Proc. Int. Conf. High Perform. Comput. Netw. Stor. Anal.*, 2020, pp. 1–15.
- [37] Y. Xu, P. Cheng, Z. Chen, M. Ding, Y. Li, and B. Vucetic, "Task offloading for large-scale asynchronous mobile edge computing: An index policy approach," *IEEE Trans. Signal Process.*, vol. 69, pp. 401–416, 2021.
- [38] M. Özel. *Drone Dataset (UAV)*. Accessed: 2025. [Online]. Available: <https://www.kaggle.com/datasets/dasmehdixtr/drone-dataset-uav>



Yang Fu received the B.S. degree from North China Electric Power University in 2022, where he is currently pursuing the Ph.D. degree with the School of Electrical and Electronic Engineering. His research interests include resource allocation in smart grid communications, space-air-ground integrated networks, vehicular networks, and the Internet of Things.



Peng Qin (Member, IEEE) received the B.S. and Ph.D. degrees in information and communication engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2009 and 2014, respectively. From 2012 to 2013, he was a Visiting Scholar with the University of Victoria, Victoria, BC, Canada. He is currently an Associate Professor with the School of Electrical and Electronic Engineering, North China Electric Power University. His research interests include resource allocation in the Internet of Things, network intelligence, smart grid communications, space-air-ground integrated networks, and vehicular networks. He was a recipient of the International Communications Signal Processing and Systems Conference Best Paper Award; and the International Conference on Artificial Intelligence in China Best Paper Award in 2019, 2020, 2021, and 2022.



Yifei Wang is currently pursuing the M.S. degree with the School of Electrical and Electronic Engineering, North China Electric Power University. His research interests include resource allocation in smart grid communications, space-air-ground integrated networks, vehicular networks, and the Internet of Things.



Liming Chen received the B.S., M.S., and Ph.D. degrees in electrical engineering from Harbin Institute of Technology, Harbin, China, in 2008, 2010 and 2014, respectively. He is currently a Researcher with the Electric Power Research Institute, China Southern Power Grid, China. His research interests include power distribution wireless communication, the IoT, cyber security, and smart distribution terminals.



Mengyao Li received the B.S. degree from North China Electric Power University in 2023, where he is currently pursuing the M.S. degree with the School of Electrical and Electronic Engineering. His research interests include space-air-ground integrated networks and wireless communication network resource optimization.

Xiongwen Zhao, photograph and biography not available at the time of publication.